
NaN-Propagation: A Novel Method for Sparsity Detection in Black-Box Computational Functions

Peter Sharpe

Massachusetts Institute of Technology

Abstract

Sparsity detection in black-box functions enables significant computational speedups in gradient-based optimization through Jacobian compression, but existing finite-difference methods suffer from false negatives due to coincidental zero gradients. These false negatives can silently corrupt gradient calculations, leading to difficult-to-diagnose errors. We introduce NaN-propagation, which exploits the universal contamination property of IEEE 754 Not-a-Number floating-point values to trace input-output dependencies through floating-point numerical computations. By systematically contaminating inputs with NaN and observing which outputs become NaN, the method reconstructs conservative sparsity patterns that eliminate false negatives. We demonstrate the approach on an aerospace wing weight model, achieving a $1.52\times$ speedup while detecting dozens of dependencies missed by conventional methods – a significant improvement since gradient computation is the bottleneck in many optimization workflows. The technique leverages IEEE 754 compliance to work across programming languages and math libraries without modifying existing black-box codes. Advanced strategies including NaN payload encoding enable faster-than-linear time complexity, improving upon existing black-box sparsity detection methods. Practical algorithms are also proposed to mitigate challenges from branching code execution common in engineering applications.

Adapted for independent reading from a chapter of the author’s Ph.D. thesis.

1 Introduction

Modern engineering design increasingly relies on multidisciplinary design optimization (MDO) frameworks that integrate diverse and complex computational models, as outlined by Martins & Ning (2021). Typically, these frameworks use gradient-based optimization, since gradient-free methods are computationally intractable as problem dimensionality increases due to the curse of dimensionality. In these gradient-based optimization workflows, the computational bottleneck is almost always in computing the gradients of both the objective function and constraint equations.

Many recent frameworks leverage advanced computational techniques such as automatic differentiation (AD) for this gradient computation, as it offers fundamentally better scaling (and accuracy) compared to traditional finite-difference methods as shown by Griewank (1988); Rackauckas (2021a). This enables efficient navigation of high-dimensional design spaces that would otherwise be intractable with traditional design approaches, as demonstrated by Sharpe (2024; 2021).

However, a fundamental challenge arises when these MDO frameworks must incorporate legacy or proprietary analysis codes that cannot be modified or instrumented for automatic differentiation. These black-box functions – computational models where internal implementation details are inaccessible – are quite common in engineering practice, particularly when progressing beyond the conceptual design stage. Examples include commercial

computational fluid dynamics (CFD) solvers, finite element analysis (FEA) packages, vendor-supplied component models, and legacy codes written in older programming languages or compiled libraries.

The integration of black-box functions into gradient-based optimization workflows necessitates using techniques such as finite differencing or the complex-step differentiation method of Martins et al. (2003), which do not rely on code introspection. While these methods can provide the necessary gradient information, they incur significant computational overhead: a function with n inputs requires $n + 1$ function evaluations (forward differences) or $2n$ evaluations (central differences) to compute its complete Jacobian. For high-fidelity engineering analyses where individual function evaluations may require minutes to hours of computation, and the number of inputs may correspond to thousands of possible design variables, this overhead becomes prohibitive.

1.1 The Role of Sparsity in Gradient Computation

The computational burden of black-box gradient evaluation can be substantially reduced through sparsity exploitation. In engineering analysis, most input-output relationships exhibit structural sparsity—many inputs do not influence certain outputs due to the underlying physics and mathematical formulation. For instance, consider a classical static structural analysis using an Euler-Bernoulli cantilevered beam model as described by Drela (1999). Here, any local forces applied near the beam’s root have no influence on the internal stress near the beam’s free tip – the influence dependency is purely one-directional: tip to root.

Knowledge of this sparsity pattern enables Jacobian compression through the graph coloring algorithms described by Gebremedhin et al. (2009; 2005). When we know that certain input variables do not interact in their influence on any output, their gradients can be computed simultaneously using appropriately constructed perturbation vectors. This can reduce the number of required function evaluations from $\mathcal{O}(n)$ to as few as $\mathcal{O}(\chi)$, where χ is the chromatic number of the sparsity pattern’s column intersection graph—often much smaller than n for engineering problems.

The potential impact of sparsity exploitation is substantial, though problem-dependent. In particularly well-suited cases, like trajectory optimization via direct collocation transcription as described by Kelly (2017a;b), evaluation of the gradient of a function with n inputs can be performed in $\mathcal{O}(1)$ time. While other cases may not see such a dramatic reduction, any improvement in accelerating the bottleneck step of gradient computation is quite helpful.

1.2 Existing Approaches

Currently, the most common approach to obtain the sparsity pattern of a black-box function is a gradient-based heuristic method. First, the (dense) Jacobian is constructed via finite differencing, evaluated at some particular point in the input space that is assumed to be representative. (In the context of an optimization framework, this input point usually corresponds to the initial guess.) Then, a critical assumption is made: it is assumed that any zero entries in this evaluated Jacobian are indicative of a zero entry in the true Jacobian – in other words, sparsity seen at one point indicates sparsity at all points. Due to this assumption, the sparsity pattern constructed with this method is only an estimate, not a guarantee.

The fact that this reconstructed sparsity pattern is only an estimate is inevitable for black-box functions. To illustrate why, consider a piecewise function, where input x_j influences output f_i when x_j is within a certain interval but not outside of it. It is possible to construct a pathological black-box function where this region of influence is arbitrarily small, such that it will nearly never be detected by direct sampling of the Jacobian. Such cases will result in a false negative in the estimated sparsity pattern: cases where x_j and f_i are believed to be independent, when in reality they are not.

This kind of branching code execution is perhaps the most common cause of such false negatives (often via conditional expressions) in the context of engineering analysis. However,

there are other relevant failure modes as well. One possible cause of false negatives is if the representative input point yields a partial derivative of zero only due to coincidence, rather than due to mathematical structure. One simple illustrative example is the function $f(x) = x^2$, if the evaluation point of $x = 0$ is chosen. Here, a central finite difference will see zero gradient and incorrectly infer no dependency. On most functions of practical interest, such points are usually rare within the input space¹. However, in an optimization context, the distribution of user-specified initial guesses is not uniform, and in fact adversarial: users will preferentially supply near-optimal initial guesses, which have near-zero gradients (if the problem is unconstrained). Because of this, and as demonstrated later in Section 6, this effect can contribute to a non-negligible rate of false negatives in practice.

False positives, where x_j and f_i are believed to be related when they are not, are also possible, though rarer. A practical example where this could occur is with a black-box function that has truncation error, perhaps due to a nonlinear solver or numerical integrator that uses a value-dependent convergence criterion. In such cases, changing one input may alter the number of iterations required for convergence. This can cause changes in the truncation error of all outputs—even those that are mathematically-unrelated to that input. Thus, changes in this truncation error (i.e., a differing number of iterations) may yield a nonzero Jacobian entry, even if the math that the code aims to represent has no such dependency. Other false positive scenarios, such as stochastic functions, are possible but relatively uncommon in engineering analysis. In either the false-negative or false-positive case, an interesting observation is that the sparsity of the function in code may not match the sparsity of the mathematical model the code aims to represent.

For reasons related to Jacobian compression (described later via demonstration in Section 2.3), the harm caused by a false-negative and a false-positive are very unequal – false negatives are far worse. The ultimate effect of a false-positive is that subsequent gradient calculations will be slightly slower, though still numerically correct. On the other hand, the ultimate effect of a false-negative is that gradient calculations can be incorrect, due to erroneous Jacobian compression. Worse yet, these incorrect results can occur silently. In the context of design optimization, an incorrect gradient might only be detected much later in the development effort, when an optimizer consistently cannot converge (as the KKT conditions cannot be satisfied). By this point in the code development process, the combined optimization model is usually much more complex, and finding and eliminating an incorrect-gradient error becomes enormously tedious. Because of this, a new method for black-box sparsity estimation that eliminates false-negative failure modes offers significant value to the end-user, even if it comes at the cost of false-positive failure modes. In other words, the focus of black-box sparsity estimation within this context should be to build up a conservative estimate of the sparsity pattern that trades away specificity in favor of sensitivity.

Recently, Hill & Dalle (2025); Rackauckas (2021b) have developed operator-overloading-based approaches to sparsity detection that do not suffer from this local-to-global problem. However, the increased robustness of such methods comes at the cost of generality: an overloading-based sparsity detection framework can only be used on code that uses a supported math library and programming language (in both of the cited examples, Julia). Therefore, these methods cannot currently be applied to general black-box functions, which may be in any language.

1.3 Contributions and Paper Organization

This paper’s primary contributions are:

1. A novel sparsity detection technique that eliminates the coincidental zero gradient failure mode of existing methods
2. Theoretical analysis and practical demonstration of the method’s advantages and limitations

¹In most practical functions $\vec{f}(\vec{x}) : \mathbb{R}^m \rightarrow \mathbb{R}^n$, the dimensionality of all manifolds where $\partial f_i / \partial x_j = 0$ is less than the dimensionality of the input $\vec{x}$. In such cases, almost all points will be free of coincidental zeros.

-
3. Advanced algorithms including NaN payload encoding for improved computational complexity
 4. Specialized strategies for engineering analysis functions with branching execution paths
 5. Comprehensive evaluation on a representative aerospace engineering application

The remainder of this paper is organized as follows: Section 2 introduces the core NaN-propagation technique with a detailed demonstration. Section 3 analyzes computational performance considerations. Sections 4 and 5 examine potential limitations and mitigation strategies. Section 6 provides quantitative comparison with existing methods. Section 7 presents advanced algorithmic improvements. Section 9 summarizes key findings and discusses future directions.

2 NaN-Propagation: Method and Demonstration

This manuscript introduces a novel technique that we call NaN-propagation to trace these input-output dependencies through black-box functions. The name refers to “not-a-number” (NaN), which is a special value in floating-point arithmetic that is often used to represent undefined, missing, or otherwise unrepresentable values. The proposed technique exploits the fact that Not-a-Number (NaN) values are universally propagated through floating-point numerical computations; colloquially, they tend to “contaminate” any calculation that is given a NaN input. For example, a dyadic² operation with just one NaN operand will return NaN. This behavior is explicitly part of math library APIs that follow the IEEE 754 standard established in 1985. Because nearly all floating-point math libraries created since the advent of IEEE 754 follow it, this presents a fascinating way to potentially trace sparsity in a way that is essentially independent of math library or programming language. By systematically and deliberately contaminating inputs to a black-box function with NaN values and observing which outputs become NaN, a provably-conservative estimate of the sparsity pattern can be reconstructed.

As a high-level comparison against the existing technique described in this section, this proposed new technique:

- Eliminates a false-negative failure mode, namely coincidental zero gradients.
- Adds a new false-positive failure mode, for functions with internal mathematical cancellation.
- Is either equivalent in runtime speed, or potentially much faster due to short-circuiting operators. This consideration is primarily determined by behavior of the math library used by the black-box function.
- Is compatible with most black-box functions, but not all, due to possible internal handling of NaN values. However, this incompatibility is usually easily and immediately detected, allowing the user to fall back to the existing technique.

This new technique, and strategies and mitigations around these four high-level tradeoffs, are the focus of the remainder of this work.

2.1 Example Black-Box Function: Wing Weight Model in TASOPT

Some of the more nuanced details of the proposed NaN-propagation technique are most clearly explained through demonstration. To do this, we leverage one of the constituent submodels within TASOPT, an aircraft design optimization suite for tube-and-wing transport aircraft developed by Drela (2010). The submodel we use is a wing weight model³, with the major modeling considerations shown in Figure 1. The model is relatively detailed, with 38 inputs and 37 outputs. Inputs include geometric parameters, material properties, and configuration-level decisions (e.g., the number and location of engines, or the presence

²referring to a function that accepts two input parameters

³In TASOPT, this model is named *surfw*.

and dimensions of a strut). Outputs are the weights of various wing components, metrics describing the weight distribution, stresses at key points, and other quantities of interest.

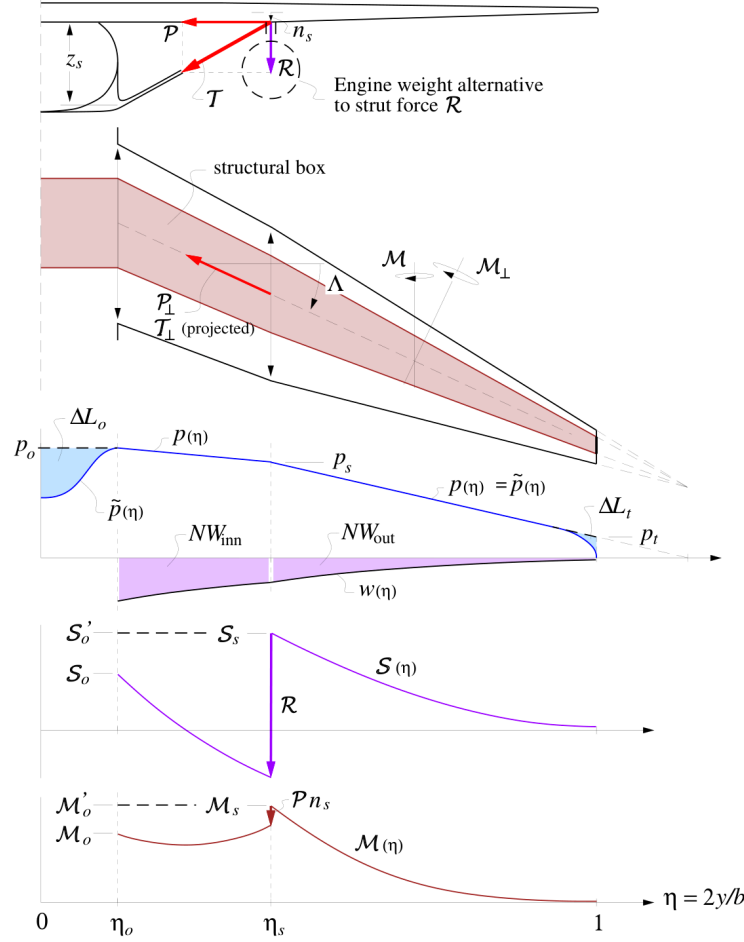


Figure 1: Illustration of TASOPT’s wing weight model, reproduced from Drela (2010). The model uses an Euler-Bernoulli beam model to compute shear and moment distribution, and accepts a relatively large set of geometric parameters as inputs.

For the purposes of this demonstration, the code implementation of this model is accessed through TASOPT.jl, which is a transpilation of the original Fortran code into Julia performed by Prakash & Gomez (2022). We aim to estimate the sparsity pattern of this function from within Python while accessing this model only through a Python-Julia interface, making this a true black-box function written in an entirely separate programming language.

2.2 Sparsity Evaluation

The first step in the NaN-propagation technique is to obtain a representative input. In the context of an optimization problem, the initial guess provides a natural choice:

```
input_vector = [114115.099, 37.533, 10.697, ..., 2700.0, 817.0]
```

Next, we create a set of “contaminated” inputs, essentially by merging this input with one-hot encoded NaN values:

```
contaminated_input_vectors = [
    [ NaN, 37.533, 10.697, ..., 2700.0, 817.0],
    [ 114115.099, NaN, 10.697, ..., 2700.0, 817.0],
    [ 114115.099, 37.533, NaN, ..., 2700.0, 817.0],
    ...,
    [ 114115.099, 37.533, 10.697, ..., NaN, 817.0],
    [ 114115.099, 37.533, 10.697, ..., 2700.0, NaN],
]
```

Finally, we evaluate the black-box function at each of these contaminated inputs, and observe which outputs become NaN. For example, the output for one such contaminated input is shown below:

```
contaminated_outputs[18] = [NaN, 3475012.2, NaN, ..., 170545.5, NaN]
```

This simple procedure yields the complete bipartite graph of which inputs and outputs are linked, and this can be interpreted as a sparsity pattern. For the demonstration case study, the sparsity pattern is shown in Figure 2.

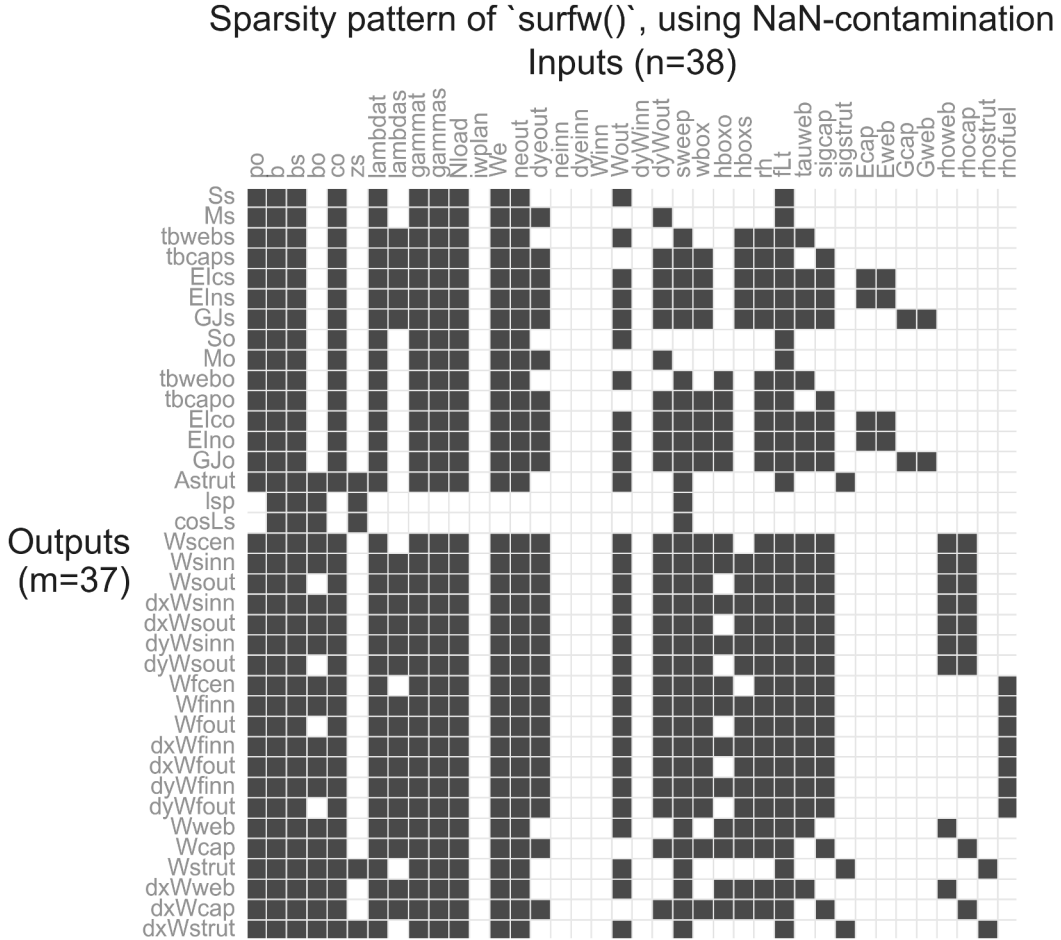


Figure 2: Sparsity pattern of the TASOPT wing weight model, estimated using NaN-propagation. In this visualization, gray squares indicate nonzero entries in the Jacobian, while white squares indicate zero entries.

A key result is that this NaN-contamination technique eliminates the possible false-negative failure mode of coincidental zero gradients that can occur with existing gradient-based methods. Consider the example case discussed in Section 1.2, where the sparsity of the function $f(x) = x^2$ is traced at $x = 0$. Here, a NaN-contamination technique will still detect the dependency between x and $f(x)$, while existing methods will not. Because the potential consequence of a false-negative is so large, this represents a significant advantage over existing methods.

2.3 Jacobian Compression

This sparsity pattern can be used to enable gradient computation accelerations via simultaneous evaluation. More precisely, this involves Jacobian compression across columns, since the gradients of this black-box will later be obtained with finite-differencing (“forward-mode”-like behavior). Briefly, the steps to achieve this are:

1. Convert the sparsity pattern into an undirected graph representation where each node corresponds to an input (i.e., a column in Figure 2), and the presence of each edge indicates that the corresponding pair of inputs has overlapping sparsity. Conveniently, the adjacency matrix of this graph can be obtained by computing the Gramian of the sparsity pattern. If the sparsity pattern is represented as the binary matrix S , then the adjacency matrix of the graph is the binary matrix $S^T S \neq 0$.
2. Apply a vertex coloring algorithm to this graph. This is a well-studied problem with many efficient approximate algorithms available, as discussed by Kubale (2004).

Visually, the result of this process can be shown with the compressed Jacobian of Figure 3. Here, several columns show a combination of multiple inputs, and gradients with respect to these inputs can be safely computed simultaneously due to structural independence. The improvement in gradient computation speed is case-dependent, but in this demonstration, a reduction from 38 inputs to 25 effective inputs is achieved. Because gradient computation runtime scales linearly with the number of inputs in a finite-difference (or complex-step) context, this means that all future gradient calculations are roughly $38/25 \approx 1.52\times$ faster than a dense Jacobian construction.

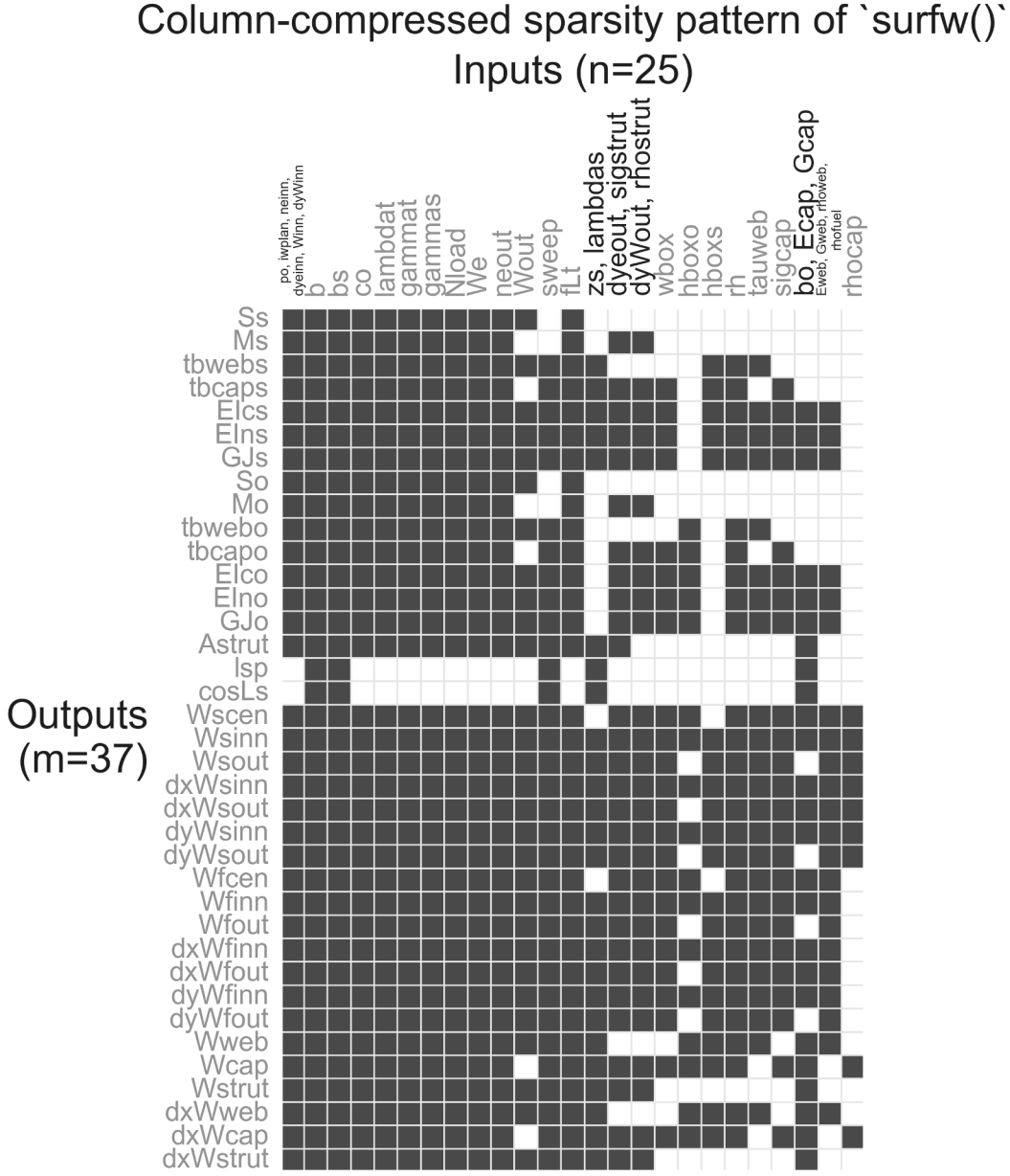


Figure 3: Compressed sparsity pattern of the TASOPT wing weight model, obtained by applying vertex coloring to the sparsity pattern of Figure 2.

3 Computational Performance Analysis

This gradient acceleration comes at a small upfront computational cost, since performing the sparsity trace requires a number of function evaluations equal to the number of inputs. This is true for both existing methods (described in Section 1.2) and for NaN-contamination. This cost is nearly always worthwhile in the context of design optimization: for the TASOPT wing model, a sparsity trace “pays for itself” in runtime if the subsequent optimization process requires more than three gradient evaluations.

In some cases, the runtime of a NaN-contamination-based sparsity trace will be essentially equal to that of standard finite-difference-based sparsity estimate. Intuitively, this makes sense, because both approaches require the same number of black-box function evaluations.

However, in other cases, NaN-contamination can actually be significantly faster due to operator short-circuiting. In this context, short-circuiting refers to operators that will check for NaN input values before performing computation; if any are found, the computation is skipped and a NaN is immediately returned instead. Programming languages and math libraries vary widely in whether and how they implement NaN-short-circuiting, so the magnitude of this potential speed-up is highly case-dependent.

4 Potential Limitations and Failure Modes

While NaN-contamination eliminates a notable false-negative failure mode compared to existing methods, it also has several possible pitfalls that merit discussion. Some of these pitfalls are shared with existing methods, while others are unique to this technique.

4.1 Mathematical False-Positives

One limitation of NaN-contamination is that it can introduce new false-positive failure modes. To illustrate one possible mechanism, consider a scenario where we attempt to NaN-propagate through a black-box function written in code as $f(x) = x - x$. If x is real-valued, then the correct result of a sparsity trace is that the input x and the output $f(x)$ are structurally unrelated. However, a NaN-contamination technique will indicate a possible dependency here, as NaN values do not subtractively cancel. In contrast, existing gradient-based methods will correctly identify independence. More generally, this kind of false-positive can occur in any function where self-cancellation leads to structural independence, such as $f(x) = \sin(x)^2 + \cos(x)^2$ or $f(x) = x^0$.

In some ways, this false-positive is an unavoidable result of the same properties that allow NaN-contamination to eliminate false negatives due to coincidental zero gradients. In theory, global knowledge of this function as a computational graph could allow this self-cancellation to be detected and avoided. However, this would require a level of introspection that is fundamentally incompatible with the black-box nature of the function.

4.2 Internal NaN Handling

While NaN-propagation does allow sparsity detection of many black-box functions, including those in other programming languages, there are some cases where it cannot be employed. In particular, black-box functions that internally handle NaN values may not allow propagation. Most commonly in this scenario, the function will instead raise an error and halt the computation. This behavior is easily and immediately detected, and the sparsity detection routine can fall back to existing gradient-based methods in these cases.

A more serious false-negative failure mode can occur if the program actively overwrites NaN outputs without raising an error, as this can result in false negatives in the sparsity trace. Fortunately, this overwriting behavior is quite rare in engineering analysis code today. The main way this can occur is if the black-box code returns special flag values (sometimes called magic numbers) to signal invalid computation, though this is generally considered an anti-pattern and avoided in numeric code. Even in code that has this overwriting behavior upon seeing a NaN result, most codes will at least raise a warning to the user.

Another possible false-positive failure mode can occur if operators within the black-box are overly aggressive about propagating NaN values. For example, consider a simple matrix-vector multiplication of the form $\vec{f}(\vec{x}) = \mathbf{A}\vec{x}$. If a single element of the matrix A_{ij} is NaN, this should result in a single NaN output f_i , while the remaining elements are unaffected. Most programming languages and math libraries (e.g., NumPy, Julia) implement this behavior, allowing an accurate sparsity trace. However, some older libraries may aggressively contaminate and yield a NaN output for the entire output vector, which effectively results in false-positives.

4.3 Branching Code Execution

Finally, branching code execution remains a challenge for NaN-contamination-based sparsity tracing, just as it is with existing methods. An example of this can be shown with the wing weight demonstration problem described in Section 2. Here, the input variable `iwplan` is essentially an enumerated type that controls the wing configuration. Specifically:

- `iwplan = 0` or `iwplan = 1` corresponds to a cantilever wing without a strut.
- All other values of `iwplan` correspond to a wing with a strut.

Perhaps unsurprisingly, the sparsity pattern of the function becomes materially different if the wing configuration is changed by adding a strut. This is shown in Figure 4.

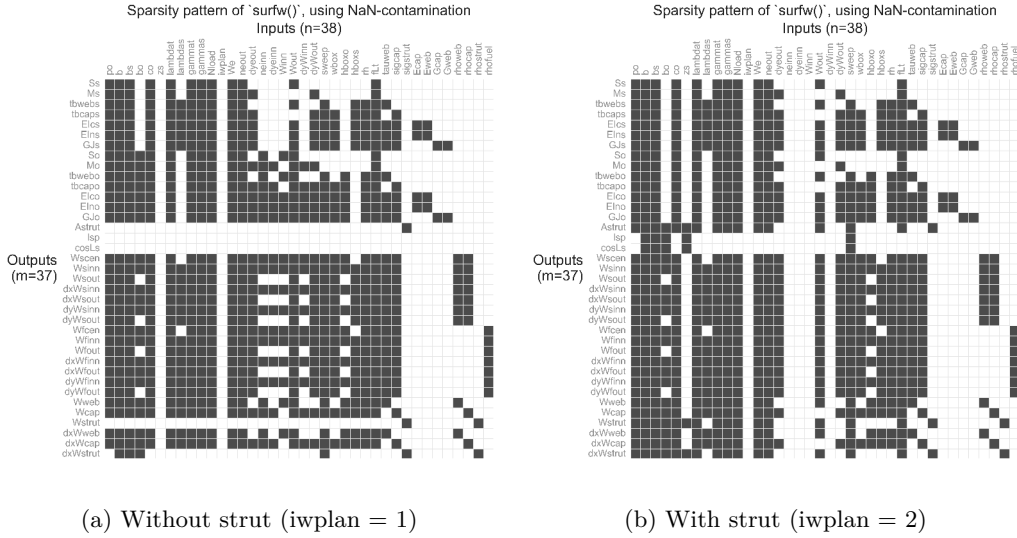


Figure 4: Sparsity pattern of the TASOPT wing weight model, estimated using NaN-propagation, as the presence of a strut is varied by changing an input variable. Sparsity patterns differ by the presence or absence of interactions across three rows and columns of the matrix.

With respect to branching code execution, NaN-contamination based strategies may have an advantage over existing methods due to the presence of static conditional operators. These operators, which are offered in several math libraries, allow for conditional statements to be represented at an operator level rather than a language level. (In many languages, this is a ternary operator with a name similar to `ifelse`, `cond`, or `where`.) In such operators, both branches of the conditional are evaluated, yet only one is returned. This contrasts with language-level conditionals (e.g., traditional `if` statements), where the actual path of code execution changes. In some math libraries (e.g., CasADi as described by Andersson et al. (2019)) static conditionals allow NaNs to propagate through regardless of the condition—if either branch evaluates to NaN, the result is a NaN even if the NaN branch would not ordinarily be taken. Because of this, NaN-contamination-based sparsity tracing can potentially handle branching code execution better than existing options, depending on the math library used by the black box.

5 Mitigating Branching Code Execution for Black-Box Engineering Analyses

While branching code execution inherently poses problems for black-box sparsity tracing in the general sense, there are heuristic strategies that can be used to improve performance

in practical cases. In particular, we look at strategies that tend to work well specifically in the context of engineering analysis code. In these cases, code branching often corresponds to variables that either change the design configuration or the mode of an analysis. The `iwplan` variable discussed in Section 4.3 is an example of this. With this in mind, we can make two observations specific to engineering analysis code that can be exploited.

First, inputs that trigger branching code execution – collectively, “flag inputs” – are often represented as discrete data types (e.g., integers, booleans, or strings) rather than continuous types (floating-point values). Because of this, knowledge of the input data types would allow some inferences to be drawn about which inputs are likely to trigger branching execution. In some cases, users may be able to manually mark flag inputs, either by leveraging domain knowledge or by inspecting known type-stable code⁴. Of course, in the general case of a black-box function, this ability to inspect a function’s type signature (either manually or dynamically) is not guaranteed. However, one possible option is to perform automatic type inference at runtime based on the types that constitute the initial guess.

Secondly, we can also take advantage of the fact that, in many cases, the value of flag inputs remains fixed during a single optimization study. For example, the design configuration may be kept fixed in a sizing study, or an analysis may only be run under certain assumptions. In such cases, the potential damage of a false negative is nullified, because the model is never evaluated on the other code branch.

One possible solution that takes advantage of both of these observations is to use a greedy approach. Concretely:

1. Run an initial sparsity trace on the black-box model using the initial guess as the input.
2. Every time a new value is observed on a discrete-typed input, re-run the sparsity trace.
3. Take the result of this sparsity trace, and compute the union with all previously-observed sparsity patterns for this black-box model. Use this new sparsity pattern for Jacobian compression in subsequent evaluations.

A visual illustration of this method is given in Figure 5. Here, the sparsity pattern is iteratively refined as new values are observed on the discrete-typed input.

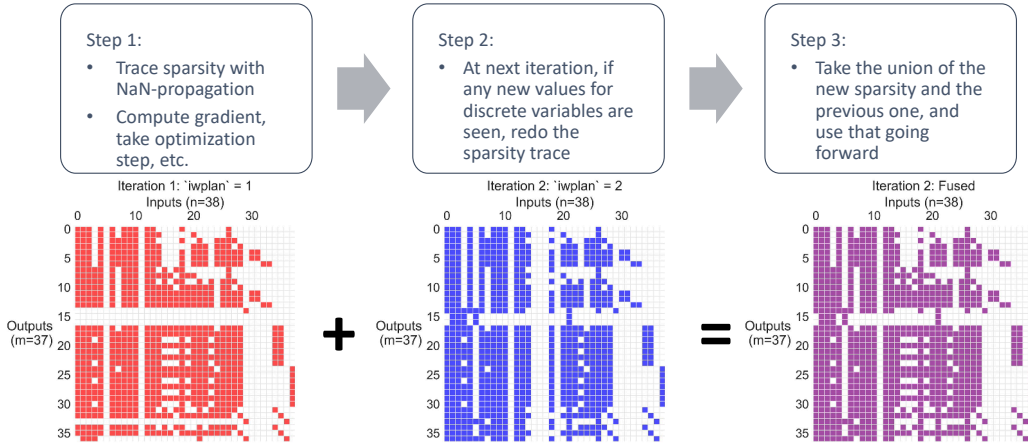


Figure 5: Illustration of a greedy algorithm for iteratively estimating the sparsity pattern of black-box models, in the presence of branching code execution. This approach is intended for use with engineering analysis models, and the process is applied to the example problem of Section 2.

⁴e.g., if the black-box function is written in a statically-typed language, and the user can view at least the function signature

It should be emphasized that this algorithm is only a heuristic, and piecewise branching on continuous variables remains problematic. However, this strategy still offers substantial practical improvements over existing techniques that use a sparsity pattern purely based on Jacobian evaluation at the initial guess.

6 Comparison with Existing Methods

In summary, the main advantage of the NaN-propagation technique is that it eliminates a key false-negative failure mode of existing gradient-based sparsity detection methods. To compare this more precisely, we can show a side-by-side comparison of the sparsity patterns obtained by each method for the TASOPT wing weight model. This is given in Figure 6. In this example, the initial guess used for Jacobian evaluation is taken from the TASOPT-supplied default for aircraft design optimization. Subfigure 6b reveals dozens of false negatives (marked as **X**) that are not captured by existing methods, but are detected with NaN-propagation. This clearly demonstrates the potential utility of NaN-propagation for sparsity-tracing black-box code functions.

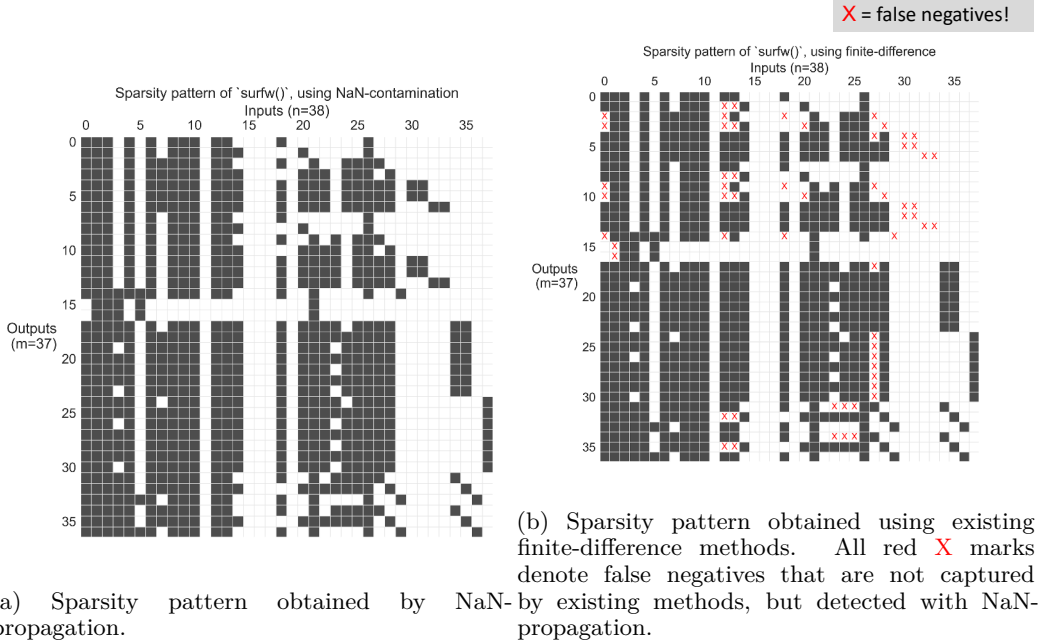


Figure 6: Comparison of sparsity patterns obtained by NaN-propagation and existing finite-difference methods for the TASOPT wing weight model. Existing methods lead to false negatives via coincidental zero gradients, which can lead to costly mistakes during Jacobian compression.

7 Advanced Algorithmic Strategies

This paper has introduced the core NaN-propagation technique, along with its benefits over existing methods and potential limitations. In this section, we introduce several advanced strategies that can be used to further improve the performance of NaN-propagation-based sparsity tracing.

7.1 NaN Payload Encoding

One possible extension is based on a recognition that not all NaN values are identical, and there are multiple unique binary representations that are interpreted as NaN values. Careful use of these values can essentially allow the encoding of additional information to aid sparsity

tracing. To illustrate this, consider the binary representation of a single-precision (32-bit) floating-point NaN value, following the IEEE 754 standard:

$$\text{NaN} = \text{x111 1111 1xxx xxxx xxxx xxxx xxxx}$$

where:

0, 1 = bit values

x = any bit (with the edge-case exception that not all trailing bits can be zero, or the value is interpreted as ∞)

Regardless of the value of the x bits, this value will be interpreted as a NaN. Therefore, single-precision NaN representations allow us to encode nearly 22 bits of information (as there are $2^{22} - 2$ unique NaN values) into these x bits, which we call the payload. In double-precision (64-bit) floating point arithmetic, which is more common in engineering analysis, this payload is even larger at nearly 51 bits.

Crucially, these unique NaN payloads are propagated through mathematical operations. Therefore, we can “tag” individual inputs with unique NaN payloads, and observe which payloads are present in the outputs. Because NaN values are no longer fungible, multiple columns of the sparsity pattern can be simultaneously traced within a single function evaluation. This presents a novel and fundamental theoretical improvement in the time-complexity of black-box sparsity tracing, since the number of function evaluations can now be independent of the number of inputs.

An immediate natural follow-up question to ask is how NaN values are propagated through dyadic operations: in cases where a function receives multiple payload-encoded NaN arguments, it is not obvious what the payload of the output will be. In general, most programming languages and math libraries will simply propagate one of the two arguments unmodified, and ignore the other. However, which argument is preferred varies. For example, in Python, the addition, subtraction, and multiplication operators propagate the first NaN input, while the division and exponentiation operators propagate the second NaN input.

Because of this effect, care must be taken when using NaN payload encoding. To demonstrate this, consider a dyadic black-box function of the form $f(x_1, x_2)$. Suppose we perform a sparsity trace by evaluating the function at $x_1 = \text{NaN}_1$, $x_2 = \text{NaN}_2$, with subscripts representing that these are uniquely identifiable NaN values. If the output of function f is NaN_1 , this indicates that f is surely a function of x_1 , but the sparsity with respect to x_2 is unknown. Likewise, the opposite is true if the output is NaN_2 . In such cases, these simultaneous column evaluations of the sparsity pattern can be thought to “shadow” one another: a sparsity “hit” on one column precludes receiving any information about the other column(s). Because of this shadowing effect, the time complexity of NaN-contamination with payload encoding is not quite $\mathcal{O}(1)$, though it can still be better than the $\mathcal{O}(N)$ of existing approaches (discussed later). Regardless, the real benefit associated with payload encoding occurs when the output f is a function of neither input, as this allows exclusion of both inputs from the sparsity pattern with a single function evaluation. This provides theoretical upper- and lower-bound cases on the method’s time complexity. To reason more intuitively and loosely, any strategy that allows us to gain information about multiple inputs from a single function evaluation should be as least as fast as standard $\mathcal{O}(N)$ time, and often faster.

This sparsity pattern construction process via NaN-payload-encoding requires an algorithm to select which combination of columns should be simultaneously evaluated next for sparsity tracing. Development of an efficient column selection algorithm is left as an area of future work. While this may at first appear to be a variant of a vertex coloring problem, the shadowing effect creates meaningful differences and a far more fascinating algorithmic challenge. To aid such future efforts, we offer some initial observations that may guide this algorithmic development for the interested reader:

1. During sparsity tracing, the state of the sparsity pattern needs to be stored in a way that captures all currently-known information: one possible representation is a trinary matrix with three possible states: 0 (no dependency), 1 (dependency), and 2 (unknown). (Other state representations that admit this partial observability

exist⁵.) Because decisions about column selection need to be made while the sparsity pattern is only partially observable, any good algorithm is inherently probabilistic. This unknown factor is also why this algorithm is not simply a degenerate case of a graph coloring algorithm.

2. A good algorithm likely benefits from some tracker of the belief state about the overall density of the sparsity pattern (i.e., the fraction of unknown entries that are likely to be dependent vs. independent). This is because the number of columns that should be evaluated simultaneously is conditional on the estimated density of the sparsity pattern. If we believe that the sparsity pattern is relatively dense, then fewer columns should be evaluated simultaneously, as the shadowing effect will be significant. Conversely, if the sparsity pattern is believed to be sparse, then we should be more aggressive about evaluating more columns simultaneously, since we will gain more information per function evaluation in expectation.

One possible way to implement this density belief state is to use a Bayesian approach, where the belief state is updated at every step based on the results of the previous function evaluations. The initial belief state could be an uninformative prior (e.g., a uniform distribution), or one could accept a user-supplied estimate. This approach has some flaws (e.g., assuming the sparsity of each entry is independent), but it presents a reasonable starting point.

3. An algorithm that selects which combination of columns should be evaluated next is inherently a combinatorial optimization problem. These are notoriously slow to solve, in the general case. However, if the black-box function is slow to evaluate (as it often is, in the case of higher-fidelity engineering analysis), it may be well worth it to solve this column-selection problem in order to minimize the number of required evaluations. In addition, there are strategies that can accelerate this combinatorial optimization problem in this context. These could include dynamic programming (since similar versions of this problem are solved sequentially), heuristic methods (e.g., simulated annealing, genetic algorithms), or continuous relaxation.

As a practical matter, one possible objective function of this optimization formulation is to maximize the expected value of information gain (measured by entropy) in the sparsity pattern, given the current belief state. In principle, this is similar to some forms of Bayesian optimization.

4. We hypothesize that best-case⁶ time complexity should approach $\mathcal{O}(\log(N))$, as measured by the number of function evaluations and where N is the number of inputs. This best-case complexity occurs in the limit of low-density sparsity patterns where the shadowing effect becomes insignificant, as the column selection procedure becomes amenable to recursive halving. A worst-case time complexity is $\mathcal{O}(N)$, which would occur in the case of a fully-dense sparsity pattern—this is identical to both existing methods and NaN-propagation without payload encoding.

7.2 Chunking

Another novel proposed acceleration, which we term “chunking”, contaminates multiple adjacent elements of the input vector with NaN values simultaneously. This is similar to the multiple seeding associated with payload NaN encoding, but its inclusion here is meant to emphasize that some benefits can be achieved even without uniquely identifying NaN values, in cases where this direct bit-manipulation may not be possible or desirable.

As an example: if pairs of adjacent inputs are seeded, this has the benefit of halving the number of function calls, at the detriment of providing overly-conservative sparsity

⁵An example would be a continuous belief state based on priors about how engineering code sparsity patterns tend to be arranged, like block-diagonality. This is more complicated, but could allow inference based on structural patterns in the partially-observed sparsity matrix. This may also have benefits in cases where branching code execution is observed, as uncertainty about sparsity can be recorded into the state and used to strategize.

⁶ignoring the trivial case where the pattern is all-zeros and this is suspected in advance

information⁷ (i.e., one with false positives). This exploits the fact that a conservative sparsity pattern can still provide some speedup during Jacobian compression, even if it is less than what might be achieved with a more accurate sparsity pattern.

It also exploits the fact that sparsity patterns in human-written code are not random. In particular, these sparsity patterns tend to have a high degree of locality (e.g., inputs at indices 3 and 4 are much more likely to share a similar sparsity pattern than inputs at indices 3 and 30). This locality is due to the fact that, in an engineering analysis with multiple submodules, humans tend to code submodule A in its entirety before implementing submodule B. Because of this, these chunked sparsity patterns using adjacent inputs can be surprisingly accurate approximations in practice.

Thus, chunking may offer significant speedups in engineering practice by exploiting typical problem structure; a computational analogy would be how an A^* graph traversal algorithm outperforms Dijkstra’s algorithm in practice despite identical worst-case complexity. This and other heuristics can significantly reduce the number of black-box function evaluations required to trace the sparsity pattern.

8 Computational Reproducibility

All source code used to generate example results in this publication is publicly available at <https://github.com/peterdsharpe/nan-propagation>.

9 Conclusions

NaN-propagation exploits the universal contamination property of IEEE 754 NaN values to detect sparsity in black-box functions. By systematically contaminating inputs with NaN and observing which outputs become NaN, the method eliminates the coincidental zero gradient failure mode that affects existing finite-difference approaches.

The technique offers several key advantages. Most importantly, it provides conservative sparsity estimates that eliminate false negatives – a critical improvement, since incorrect sparsity patterns can silently corrupt gradient calculations during Jacobian compression. On the TASOPT wing weight model, NaN-propagation detected dozens of dependencies missed by conventional methods and achieved a $1.52\times$ gradient computation speedup.

The method has limitations. It can introduce false positives in functions with mathematical self-cancellation (e.g., $f(x) = x - x$), and some black-box functions may not be compatible due to internal NaN handling. Fortunately, false positives have relatively low impact, resulting in a speed penalty but without compromising numerical correctness of gradients. Like existing methods, NaN-propagation struggles with branching code execution, though the proposed greedy algorithm provides practical improvements for engineering applications.

Advanced strategies can further improve performance. NaN payload encoding allows multiple columns to be traced simultaneously, offering faster-than-linear time complexity for sparse problems. Chunking techniques exploit the locality common in engineering code to reduce function evaluations with minimal accuracy loss, again yielding speed improvements.

The universal nature of IEEE 754 NaN behavior makes this approach broadly applicable across programming languages and computational environments. This is particularly valuable for engineering optimization, where legacy and proprietary analysis codes are common and gradient-based methods are essential for high-dimensional problems.

Future work should focus on developing optimal column selection algorithms for NaN payload encoding, creating hybrid approaches that combine multiple sparsity detection methods, and implementing the technique in popular optimization and machine learning frameworks to enable widespread adoption.

⁷since one cannot determine which NaN input caused a given output to return NaN, so neither input can be ruled out – both inputs must be assumed to be the possible cause

References

- Joel A. E. Andersson, Joris Gillis, Greg Horn, James B. Rawlings, and Moritz Diehl. CasADi: A software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 11(1):1–36, March 2019. ISSN 1867-2949, 1867-2957. doi: 10.1007/s12532-018-0139-4.
- Mark Drela. Integrated simulation model for preliminary aerodynamic, structural, and control-law design of aircraft. In 40th Structures, Structural Dynamics, and Materials Conference and Exhibit, St. Louis, MO, U.S.A., April 1999. American Institute of Aeronautics and Astronautics. doi: 10.2514/6.1999-1394.
- Mark Drela. TASOPT: Transport Aircraft System Optimization. Technical Description, March 2010.
- Assefaw H. Gebremedhin, Arijit Tarafdar, Alex Pothén, and Andrea Walther. Efficient Computation of Sparse Hessians Using Coloring and Automatic Differentiation. *INFORMS Journal on Computing*, 21(2):209–223, May 2009. ISSN 1091-9856, 1526-5528. doi: 10.1287/ijoc.1080.0286.
- Assefaw Hadish Gebremedhin, Fredrik Manne, and Alex Pothén. What Color Is Your Jacobian? Graph Coloring for Computing Derivatives. *SIAM Review*, 47(4):629–705, January 2005. ISSN 0036-1445, 1095-7200. doi: 10.1137/S0036144504444711.
- Andreas Griewank. On Automatic Differentiation. In *Mathematical Programming: Recent Developments and Applications*. Argonne National Laboratory, Argonne, Illinois, 1988.
- Adrian Hill and Guillaume Dalle. Sparser, Better, Faster, Stronger: Sparsity Detection for Efficient Automatic Differentiation, June 2025.
- Matthew Kelly. An Introduction to Trajectory Optimization: How to Do Your Own Direct Collocation. *SIAM Review*, 59(4):849–904, January 2017a. ISSN 0036-1445, 1095-7200. doi: 10.1137/16M1062569.
- Matthew P. Kelly. Transcription Methods for Trajectory Optimization: A beginners tutorial. arXiv:1707.00284 [math], July 2017b.
- Marek Kubale (ed.). Graph Colorings. Number 352 in *Contemporary Mathematics*. American Mathematical Society, Providence, R.I, 2004. ISBN 978-0-8218-3458-9.
- Joaquim R. R. A. Martins and Andrew Ning. *Engineering Design Optimization*. Cambridge University Press, 1 edition, November 2021. ISBN 978-1-108-98064-7 978-1-108-83341-7. doi: 10.1017/9781108980647.
- Joaquim R. R. A. Martins, Peter Sturdza, and Juan J. Alonso. The complex-step derivative approximation. *ACM Transactions on Mathematical Software*, 29(3):245–262, September 2003. ISSN 0098-3500, 1557-7295. doi: 10.1145/838250.838251.
- Prashanth Prakash and Nicolas Gomez. TASOPT.jl, 2022. GitHub repository. <https://github.com/MIT-LAE/TASOPT.jl>.
- Christopher Rackauckas. Engineering Trade-Offs in Automatic Differentiation: From TensorFlow and PyTorch to Jax and Julia, December 2021a.
- Christopher Rackauckas. Generalizing Automatic Differentiation to Automatic Sparsity, Uncertainty, Stability, and Parallelism, March 2021b.
- Peter D. Sharpe. AeroSandbox: A Differentiable Framework for Aircraft Design Optimization. Master’s thesis, Massachusetts Institute of Technology, 2021.
- Peter D. Sharpe. Accelerating Practical Engineering Design Optimization with Computational Graph Transformations. PhD thesis, Massachusetts Institute of Technology, 2024.